Parameter-Efficient Fine-Tuning for Visual Question Answering: A Comparative Study

Sophia Yang (301627219), Qingrui Li (301400099)

***Abstract*—Visual Question Answering (VQA) is a multimodal task where models must reason over both images and natural language questions. Recent large vision–language models such as BLIP-2 achieve strong performance by combining frozen vision encoders with large pretrained language models, but fine-tuning these models end-to-end is computationally expensive and resource-intensive. In this project, we systematically compare parameter-efficient fine-tuning (PEFT) methods for adapting BLIP-2 to VQA, evaluating their trade-offs between accuracy, memory efficiency, and training cost. We evaluate three approaches—LoRA, Bottleneck Adapters, and IA^3—applied to a frozen BLIP-2 (OPT-2.7B) backbone on the VQA v2.0 benchmark (443K training / 214K validation questions), training on a fixed 30K subset and evaluating on 3,000 validation samples. Our results show that LoRA achieves the best accuracy (47.37%), an improvement of +7.08 percentage points over the frozen baseline (40.29%). Bottleneck Adapters achieve comparable performance (46.13%, +5.84 pp) with more trainable parameters (10.73M, 0.29%). Surprisingly, IA^3 (36.78%) falls below the baseline, suggesting that its minimal parameter budget (∼0.73M, 0.019%) is insufficient to overcome the pretrained model's priors under our 30K training regime. IA^3 achieves the lowest inference latency (1108.8 ms/sample), while Adapters incur the highest overhead (2155.6 ms/sample). These findings provide empirical guidance for efficient VQA model adaptation under resource constraints.**

***Index Terms*—Visual Question Answering, Parameter-Efficient Fine-Tuning, BLIP-2, LoRA, Adapters, IA^3, Vision–Language Models.**

I. Introduction

VISUAL Question Answering (VQA) aims to answer natural language questions grounded in visual content. Unlike image classification or captioning, VQA requires more detailed reasoning across both modalities, such as recognizing objects, identifying attributes, and performing basic visual reasoning. As a result, VQA has become a standard benchmark for evaluating vision–language understanding.

Recent advances in large vision–language models (VLMs) have significantly improved VQA performance. Earlier architectures such as ViLT demonstrated that transformer-based models can jointly process vision and language without convolutional backbones or region supervision [1]. More recent models such as BLIP-2 leverage pretrained vision encoders and large language models (LLMs) to achieve strong results across a variety of multimodal tasks [2]. However, the scale of these models introduces new challenges related to training efficiency and scalability, particularly when adapting them to downstream tasks such as VQA.

Although large VLMs demonstrate impressive performance, fully fine-tuning all parameters is often impractical due to high memory and computational costs. Parameter-efficient fine-tuning (PEFT) methods such as LoRA [3], Adapters [4], and IA^3 [5] provide alternatives by updating only a small subset of parameters while keeping most of the model frozen. These methods differ in their architectural modifications: LoRA uses low-rank weight updates, Adapters insert bottleneck layers, and IA^3 applies learned scaling to activations. As a result, they vary in the number of additional trainable parameters, memory usage, and computational overhead introduced during training.

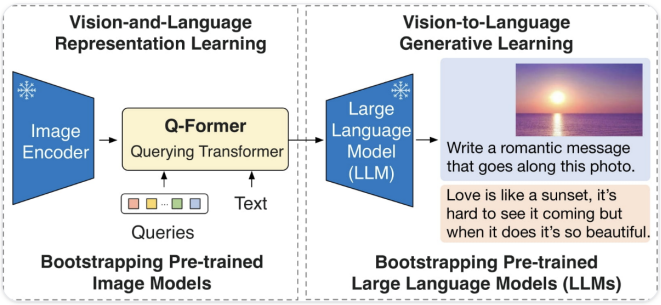

Fig. 1. Overview of the BLIP-2 architecture. A frozen vision encoder (ViT-L/14) feeds into a Q-Former, which bridges to a frozen OPT-2.7B language model. Only the Q-Former is trained during BLIP-2 pretraining.

In this project, we systematically compare these PEFT methods for adapting BLIP-2 to VQA, investigating whether they can achieve competitive performance with minimal parameter overhead, and how different methods trade off between accuracy, memory, and training efficiency. Concretely, we address the following research questions: **(1) Which PEFT method achieves the best VQA accuracy when adapting BLIP-2 under a fixed compute budget? (2) How do the methods compare in terms of performance-efficiency trade-offs, including inference latency, trainable parameter count, and training cost?** Our goal is to provide empirical guidance for efficient VQA model adaptation under resource constraints.

The practical motivation for this study is broad. Accurate, low-latency VQA systems are increasingly valuable in real-world settings: assistive technologies that describe scenes or answer questions for visually impaired users, medical imaging assistants that help clinicians query image content in natural language, autonomous robots that must answer contextual questions about their environment, and interactive educational tools that respond to student queries about visual materials. In all of these scenarios, deploying a model that is both accurate and resource-efficient is critical—full fine-tuning of billion-parameter models is impractical on edge devices or within limited cloud budgets, making PEFT an attractive and timely approach.

II. Related Work

A. Vision–Language Models

Early VLMs relied on region-based visual features extracted from object detectors, combined with language models for cross-modal reasoning. ViLT [1] simplified this pipeline by processing flattened image patches directly through a transformer, eliminating the need for convolutional backbones or region supervision. BLIP-2 [2] extends this line of work by introducing a Querying Transformer (Q-Former) that bridges a frozen vision encoder (ViT-L/14) and a frozen large language model (OPT-2.7B), achieving strong performance across a range of vision–language tasks including VQA and image captioning. By keeping both the vision encoder and the language model frozen during pretraining and training only the Q-Former, BLIP-2 substantially reduces the number of parameters that must be updated.

B. Parameter-Efficient Fine-Tuning

Several PEFT approaches have been proposed to reduce the cost of adapting large pretrained models to downstream tasks.

LoRA [3] injects trainable low-rank decomposition matrices into selected weight matrices of the transformer, typically the query and value projection layers. Since the rank r is much smaller than the original weight dimensions, LoRA introduces very few additional parameters while enabling effective task-specific adaptation.

Bottleneck Adapters [4] insert small residual modules—consisting of a down-projection, a nonlinearity, and an up-projection—within each transformer layer. Only the adapter parameters are trained, while the original weights remain frozen.

IA^3 [5] learns element-wise rescaling vectors applied to the keys, values, and feed-forward network (FFN) activations of each transformer layer. Because only these lightweight vectors are trained, IA^3 introduces far fewer parameters than either LoRA or adapters.

C. VQA Benchmark

The VQA v2 dataset [6] is a large-scale benchmark designed to reduce language-only biases by pairing each question with two visually similar images that require different answers, encouraging models to rely on visual evidence rather than language priors. Each question is associated with ten human-provided annotations, and model predictions are scored using a soft accuracy metric that accounts for inter-annotator disagreement. Recent survey work highlights VQA v2 as a standard benchmark for evaluating grounding and robustness in multimodal systems [7].

III. Methods

A. Base Model

We use BLIP-2 with an OPT-2.7B language model backbone (`Salesforce/blip2-opt-2.7b` from Hugging Face) as our base model. The model is loaded in 8-bit quantization using `bitsandbytes` to reduce GPU memory consumption. Consistent with the original BLIP-2 design, both the vision encoder (ViT-L/14) and the language model (OPT-2.7B) are kept entirely frozen across all experiments. Only the PEFT-specific parameters are trained.

TABLE I
Representative VQA v2 samples showing question types, annotation distributions, and resulting VQA scores for a correct prediction.

Question	Annotations (10 humans)	Score
Is the dog on a leash?	yes×7, no×3	1.00
What color is the fire hydrant?	red×10	1.00
How many people are there?	2×6, 3×3, 1×1	1.00
What sport is being played?	tennis×5, baseball×3, golf×2	1.00
What is on the pizza?	cheese×2, toppings×2, sauce×2, meat×2, veggies×2	0.67

B. Dataset

We train and evaluate on the VQA v2.0 dataset [6], which contains 443,757 training questions and 214,354 validation questions paired with MS-COCO images. Due to computational constraints, we sample a fixed subset of 30,000 question–image pairs from the official training set for all experiments, and evaluate on a fixed subset of 3,000 validation samples. All methods are trained and evaluated under the same data conditions to ensure a fair comparison. Each training example is formatted as:

```
``Question: {q} Answer: {a}''
```

During training, the loss is computed only over the answer tokens, with the prompt tokens masked out. During evaluation, the model generates an answer given the question prompt and the input image.

Table I illustrates representative samples from the VQA v2 dataset. Each question is paired with ten independent human annotations. The soft accuracy score rewards partial agreement: a prediction matching 3 or more annotators scores 1.0, and fewer matches yield proportionally lower scores.

C. Frozen Baseline

The frozen baseline evaluates the pretrained BLIP-2 model without any fine-tuning. No parameters are updated. This condition establishes the zero-shot performance floor for VQA v2 under our evaluation setup and serves as a reference point for all PEFT methods. To isolate the effect of answer post-processing, we additionally evaluate the frozen baseline without applying the normalization pipeline.

Table II shows verbatim BLIP-2 outputs on six validation samples. Because OPT-2.7B is a generative language model, it produces sentence-length or context-laden responses—```No, it's an orange tabby cat''`, ```2 people''`, ```As much as possible''`—rather than the concise

TABLE II
VERBATIM BLIP-2 OUTPUTS ON SIX REPRESENTATIVE VALIDATION SAMPLES (NO POST-PROCESSING APPLIED). **RAW OUTPUT** IS EXACTLY WHAT THE MODEL GENERATES; **GT** IS THE MAJORITY GROUND-TRUTH ANNOTATION; **SCORE** IS THE VQA SOFT SCORE WITHOUT NORMALISATION. ALL SIX SCORE 0.00 RAW, MOTIVATING THE NORMALISATION PIPELINE DESCRIBED ABOVE.

Question	Raw Output (BLIP-2)	GT	Score
How many people are there?	2 people	2	0.00
Is this animal a zebra?	No, it's an orange tabby cat	no	0.00
Is that a pizza?	Yes, it is.	no	0.00
What's on the pizza?	Pizza toppings	olives	0.00
What color are the rocks?	They're gray	brown	0.00
How many feet to keep back?	As much as possible	50	0.00

annotations VQA v2 expects. All six examples score 0.00 without post-processing, highlighting the necessity of the normalization pipeline described in Section III-I.

D. LoRA

We apply LoRA to the query and value projection layers (`q_proj` and `v_proj`) of the OPT-2.7B attention mechanism. The configuration uses rank $r = 8$, scaling factor $\alpha = 16$, and dropout $= 0.05$, introducing approximately 2.62M trainable parameters (0.07% of total). Training uses a learning rate of 1×10^{-4} with the AdamW optimizer.

E. Bottleneck Adapters

We implement custom bottleneck adapter modules injected after each of the 32 decoder layers in OPT-2.7B using forward hooks. Each adapter module consists of a LayerNorm, a down-projection to a bottleneck dimension of 64, a GELU activation, and an up-projection back to the original hidden dimension, followed by a residual connection. This configuration introduces approximately 10.73M trainable parameters (0.29% of total). Training uses a learning rate of 1×10^{-3} with AdamW.

F. IA^3

We apply IA^3 to the key and value projections and the FFN outputs of each OPT-2.7B decoder layer. IA^3 learns element-wise rescaling vectors for each of these components, resulting in approximately 0.73M trainable parameters (0.019%)—the smallest parameter budget among all trained methods. Training uses a learning rate of 3×10^{-3} with AdamW.

G. Training Setup

All PEFT methods are trained for 3 epochs using a per-device batch size of 2 with gradient accumulation over 4 steps, yielding an effective batch size of 8. The AdamW optimizer is used for all methods. All experiments are conducted under the same computational budget to ensure a fair comparison.

H. Implementation Challenges

Adapting a generative vision–language model to VQA introduced several non-trivial engineering challenges. First, OPT-2.7B reuses the same token ID (2, `</s>`) as both the end-of-sequence (EOS) token and the padding token in the HuggingFace tokenizer configuration. A naive implementation that masks labels at positions equal to `pad_token_id` will also mask EOS tokens, preventing the model from learning when to terminate generation. This caused runaway generation—the model producing verbose, sentence-length outputs such as ``hut for elephants to stay in when they are not'' instead of the single-word answer ``hut''. We resolved this by masking only positions where `attention_mask` $= 0$ (true padding), leaving EOS tokens in the loss:

```
labels[attention_mask == 0] = -100
```

Second, 8-bit quantization via `bitsandbytes` requires calling `prepare_model_for_kbit_training()` before attaching PEFT modules; omitting this step causes gradient dtype mismatches at training time. Third, for Bottleneck Adapters we inject modules via PyTorch forward hooks rather than modifying the model class, which requires careful hook removal after evaluation to prevent memory leaks.

I. Evaluation

We evaluate each method using the official VQA v2 accuracy metric:

$$\text{VQA Score} = \min\left(\frac{\text{matching annotations}}{3},\ 1.0\right) \quad (1)$$

1) Answer Normalization: Since all methods share the same OPT-2.7B backbone, they all produce sentence-length outputs (e.g., ``No, it's an orange tabby cat'', ``2 people'') rather than the short annotations VQA v2 expects. To ensure **fair, uniform comparison across all methods**—directly supporting the accuracy rankings in RQ1 and the efficiency analysis in RQ2—we apply an identical five-step normalisation pipeline to every prediction before scoring: (1) collapse repeated tokens; (2) truncate at the first punctuation boundary; (3) extract a leading digit (e.g., ``2 people'' $\rightarrow$ ``2''); (4) strip trailing filler after yes/no (e.g., ``No, it's an orange tabby cat'' $\rightarrow$ ``no''); (5) lowercase. Because this pipeline is applied identically to all methods, it does not alter the relative ranking between models and therefore does not bias the comparison.

To empirically quantify its necessity, we evaluated the frozen baseline without normalisation, obtaining 30.40% raw accuracy vs. 40.29% normalised (+9.89 pp; Table III, "Baseline (no norm)" row). Table II shows the raw BLIP-2 outputs that score 0.00 without normalisation; steps 3–4 of the pipeline recover the correct answer from verbose outputs such as ``2 people'' and ``No, it's an orange tabby cat''. Any residual underestimation from phrasing gaps affects all methods equally and does not alter relative rankings.

TABLE III
COMPARISON OF PEFT METHODS ON VQA V2. ALL METHODS TRAINED ON 30K SAMPLES FOR 3 EPOCHS. TRAINING TIME IS AVERAGED ACROSS EPOCHS. GPU VRAM IS PEAK USAGE DURING TRAINING.

Method	Accuracy (%)	Params (M)	Params (%)	Time / ep (h)	Total Train (h)	Latency (ms/sample)	VRAM (GB)
Baseline (no norm)†	30.40	0	0	—	—	—	—
Baseline	40.29	0	0	—	—	1449.6	—
LoRA	**47.37**	2.62	0.070	3.72	11.16	1931.8	3.95
Adapters	46.13	10.73	0.286	3.74	11.22	2155.6	3.95
IA^3	36.78	0.73	0.019	3.69	11.07	**1108.8**	3.93

†Same frozen BLIP-2 model; answer normalisation disabled. Raw accuracy: 30.40% (gain from norm: +9.89 pp).

TABLE IV
TRAINING CROSS-ENTROPY LOSS PER EPOCH FOR EACH PEFT METHOD (LOWER IS BETTER).

Method	Epoch 1	Epoch 2	Epoch 3
LoRA	0.5154	0.4487	0.4373
Adapters	0.4763	0.4283	0.3978
IA^3	0.5714	0.4397	0.4235

2) Efficiency Metrics: To assess performance-efficiency trade-offs, we report the following metrics for each method: (1) **VQA accuracy** (%), the primary task performance measure; (2) **trainable parameters** (count in millions and as a percentage of total model parameters), indicating parameter efficiency; (3) **training time per epoch** (hours), capturing the per-epoch computational cost of adaptation on 30K samples; (4) **total training time** (hours, summed over 3 epochs), reflecting the end-to-end training cost under our experimental setup; (5) **inference latency** (ms/sample), measuring the average time to produce one answer at evaluation time with batch size 8; and (6) **peak GPU VRAM** (GB) during training, reflecting hardware requirements. Together, these metrics allow us to characterise the accuracy-efficiency frontier and identify which method offers the best trade-off for practitioners operating under resource constraints.

IV. RESULTS

Table III summarises the main results across all methods evaluated on 3,000 fixed validation samples from VQA v2.

A. Training Loss Progression

Table IV reports the training loss at the end of each epoch for all three PEFT methods, measured on 30K training samples. All methods show consistent loss reduction across epochs, confirming that learning is occurring. Adapters converge to the lowest first-epoch loss (0.4763) and the lowest final loss (0.3978), reflecting their larger parameter capacity (10.73M). LoRA and IA^3 reach comparable final losses (0.4373 and 0.4235, respectively), yet their VQA accuracies diverge substantially: LoRA improves +7.08 pp over the baseline while IA^3 falls below it. This loss–accuracy gap suggests that IA^3's learned scaling vectors may overfit the training distribution while degrading the generalisation encoded in the pretrained representations.

TABLE V
QUALITATIVE PREDICTIONS ON FIVE REPRESENTATIVE VQA V2 VALIDATION QUESTIONS. ✓ = CORRECT (SCORE ≥ 0.67), × = INCORRECT (SCORE $= 0$).

Question	GT	Base	LoRA	Adapt.	IA^3
Is the dog on a leash?	yes	yes ✓	yes ✓	yes ✓	yes ✓
What color is the bus?	yellow	yellow ✓	yellow ✓	yellow ✓	yellow ✓
What is the bird sitting on?	branch	tree br. ×	branch ✓	branch ✓	tree br. ×
How many people are there?	2	2 ✓	2 ✓	3 ×	2 ✓
What color are the slippers?	black	brown ×	black ✓	black ✓	gray ×

B. VQA Accuracy

LoRA achieves the highest VQA accuracy at 47.37%, representing a +7.08 percentage point improvement over the frozen baseline (40.29%). Bottleneck Adapters follow closely at 46.13% (+5.84 pp), despite having four times more trainable parameters than LoRA. IA^3 at 36.78% falls 3.51 percentage points *below* the frozen baseline, indicating that its minimal parameter budget is insufficient to overcome the pretrained model's priors under a 30K training regime.

The convergence behaviour also differs across methods. Adapters achieve the lowest final training loss (0.3978 after epoch 3), suggesting strong fitting capacity on the answer-generation objective. LoRA converges to 0.4373 and IA^3 to 0.4235; notably, IA^3's lower final training loss than LoRA does not translate to higher VQA accuracy, indicating that the learned scaling vectors may distort the pretrained representations rather than specialising them for the task.

C. Qualitative Comparison

Table V shows side-by-side predictions from all four methods on five representative validation questions. Fine-tuned methods generally produce shorter, more annotation-aligned answers than the frozen baseline. On yes/no questions all methods perform well, while open-ended questions reveal differences: LoRA and Adapters more reliably match the expected short-form answer, whereas IA^3 occasionally reverts to verbose baseline-like outputs. The "bird" example (ground truth: `branch`) illustrates scorer underestimation—``tree branch'' is semantically correct but scores zero because it does not match any annotation exactly.

D. Efficiency Trade-offs

Training time. All three PEFT methods require nearly identical total training time: LoRA 11.16 h, Adapters 11.22 h, and IA^3 11.07 h across 3 epochs. This near-equality is surprising given that Adapters introduce 10.73M trainable parameters—roughly 15× more than IA^3 (0.73M) and 4× more than LoRA (2.62M). The explanation is that training time is dominated by the *frozen* OPT-2.7B forward and backward pass: gradients must still propagate through all 32 frozen decoder layers to reach the PEFT modules, regardless of how many PEFT parameters exist. The PEFT modules themselves contribute negligible compute. This finding has a practical implication: choosing a PEFT method with more parameters does not increase training cost under a frozen-backbone setup, so the decision should be made purely on accuracy and inference grounds.

GPU memory. Similarly, peak VRAM is nearly identical across methods (≈3.93–3.95 GB), again because the frozen backbone dominates memory consumption. The 8-bit quantization via `bitsandbytes` keeps the 2.7B-parameter OPT model within consumer GPU limits (8 GB VRAM), which would otherwise require ∼10 GB at full float16 precision.

Inference latency. Latency differs more substantially between methods. IA^3 is the fastest (1108.8 ms/sample) since its learned scaling vectors require only element-wise multiplications with no additional layers. The frozen baseline (1449.6 ms/sample) is slower due to the absence of any caching or optimisation. LoRA (1931.8 ms/sample) adds a small overhead from the $B \times A$ low-rank matrix multiplications in each attention layer. Bottleneck Adapters are the slowest (2155.6 ms/sample) because each of the 32 OPT decoder layers passes activations through an additional down-projection, nonlinearity, and up-projection at every forward pass. The latency ordering (IA^3 < Baseline < LoRA < Adapters) directly reflects the amount of additional computation each method introduces at inference time.

E. Discussion

RQ1 — Best accuracy under a fixed compute budget. LoRA answers RQ1 decisively: it achieves the highest VQA accuracy (47.37%, +7.08 pp over baseline) with only 2.62M trainable parameters. The low-rank constraint (rank 8) acts as an implicit regulariser, preventing overfitting on the 30K training set. Bottleneck Adapters are a competitive second (46.13%) but require four times more parameters without a proportional accuracy gain, suggesting diminishing returns from added capacity under our compute budget. IA^3 (36.78%) falls below the frozen baseline, showing that its minimal parameter budget is insufficient to overcome the pretrained model's priors on 30K examples alone.

RQ2 — Performance-efficiency trade-offs. Training time and GPU VRAM are nearly identical across all PEFT methods (≈11 h, ≈3.95 GB), because both costs are dominated by the frozen OPT-2.7B backbone. The key differentiator is *inference latency*: IA^3 is the fastest (1108.8 ms/sample) since its scaling vectors add no extra layers, while Adapters are the slowest (2155.6 ms/sample) due to the 32 additional bottleneck modules in the forward pass. LoRA sits in between (1931.8 ms/sample). For practitioners, this means IA^3 is preferable when inference speed is the primary constraint and some accuracy loss is acceptable, while LoRA offers the best accuracy-latency balance.

The IA^3 underperformance is the most surprising finding. Its element-wise rescaling introduces far fewer degrees of freedom than LoRA or Adapters and appears unable to learn sufficient task-specific signal from 30K examples. Extending training to more data or epochs is a natural avenue for future work.

A further source of error shared by all methods is scorer underestimation: semantically correct but differently phrased answers (e.g., ``pine tree'' vs. ground truth ``pine'') score zero. The reported accuracy figures therefore likely *underestimate* true performance for all methods uniformly.

F. Limitations

Several limitations should be noted when interpreting our results. First, our training subset (30K samples) is only 6.8% of the full VQA v2 training split, and our evaluation set (3,000 samples) is 1.4% of the validation split. While sufficient for method comparison, these sizes may not fully reflect performance on the complete benchmark. Importantly, the inclusion of a *frozen baseline*—BLIP-2 with no fine-tuning whatsoever—ensures that all comparisons remain valid even under this reduced compute budget: any PEFT method that fails to surpass the frozen baseline (as IA^3 does here) provides a clear signal that the method requires more training data or a different configuration, rather than simply more compute. The frozen baseline thus serves as a meaningful lower bound that anchors the comparison regardless of dataset scale. Second, we apply PEFT exclusively to the OPT-2.7B language model component; the ViT vision encoder and the Q-Former bridge remain entirely frozen. Applying PEFT to the Q-Former, which is the primary cross-modal alignment module in BLIP-2, could yield additional gains and is unexplored here. Third, all experiments are conducted on a single consumer GPU; results may differ under multi-GPU or mixed-precision training configurations. Finally, the answer normalisation pipeline, while necessary for fair scoring, introduces heuristics that may over- or under-credit certain predictions, slightly distorting the reported accuracy figures for all methods equally.

V. Conclusion

We presented a systematic empirical comparison of three parameter-efficient fine-tuning methods—LoRA, Bottleneck Adapters, and IA^3—applied to BLIP-2 (OPT-2.7B) for VQA v2. All methods were trained under identical conditions on 30,000 samples for 3 epochs and evaluated on 3,000 fixed validation samples.

Addressing **RQ1** (best accuracy under a fixed compute budget): LoRA achieves the highest VQA accuracy (47.37%, +7.08 pp over the frozen baseline) with only 2.62M trainable parameters (0.07%), making it the most effective method under our compute budget. Bottleneck Adapters are a competitive second (46.13%) at higher parameter cost and inference

latency. IA^3 underperforms the frozen baseline (36.78%), demonstrating that its minimal parameter budget of 0.73M may be insufficient for robust VQA adaptation without significantly more training data or epochs.

Addressing **RQ2** (performance-efficiency trade-offs): all PEFT methods consume similar GPU VRAM ($\approx$3.95 GB) and training time ($\approx$11 h), since both are dominated by the frozen backbone. Inference latency is the key differentiator—IA^3 is the fastest (1108.8 ms/sample) owing to its parameter-free inference overhead, while Adapters are the slowest (2155.6 ms/sample) due to 32 additional bottleneck modules in every forward pass.

These results suggest that for practitioners adapting large vision–language models to VQA under resource constraints, LoRA offers the best accuracy-efficiency trade-off. In future work, it would be interesting to explore scaling IA^3 to larger training sets, combining PEFT methods, or applying PEFT to additional components of the BLIP-2 architecture such as the Q-Former.

REFERENCES

[1] W. Kim, B. Son, and I. Kim, "ViLT: Vision-and-language transformer without convolution or region supervision," *arXiv preprint arXiv:2102.03334*, 2021.

[2] J. Li, D. Li, S. Savarese, and S. Hoi, "BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models," *arXiv preprint arXiv:2301.12597*, 2023.

[3] E. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2022.

[4] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. De Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, "Parameter-efficient transfer learning for NLP," in *Proc. 36th Int. Conf. Mach. Learn. (ICML)*, 2019.

[5] H. Liu, D. Tam, M. Muqeeth, J. Mohta, T. Huang, M. Bansal, and C. Raffel, "Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2022.

[6] Y. Goyal, T. Khot, D. Summers-Stay, D. Batra, and D. Parikh, "Making the V in VQA matter: Elevating the role of image understanding in visual question answering," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2017.

[7] R. Kabir, N. Haque, M. S. Islam, and Marium-E-Jannat, "A comprehensive survey on visual question answering datasets and algorithms," *arXiv preprint arXiv:2411.11150*, 2024.